

Lecture 4: Complexity of Recursive Algorithms I

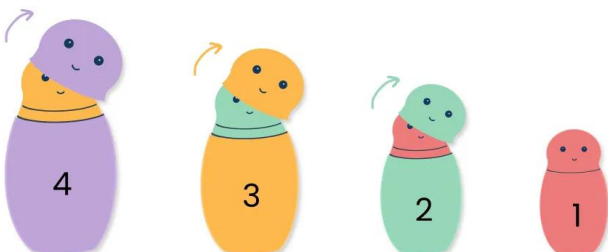
- Recursion
- Merge Sort
- Substitution method for recurrences
- Two example recursive algorithms

Recursion

- When a function calls itself

```
def my_function(n):  
    if n>0:  
        print(n)  
        my_function(n-1)
```

```
my_function(2)  
    print(2)  
    my_function(1)  
        print(1)  
        my_function(0)
```



What does this function do?

```
def main():  
    print("Here!")  
    main()  
    print("Important message!")  
  
    main()  
  
main()  
  
    print("Here!")  
    main()  
    print("Here!")  
    main()  
    print("Here!")  
    main()  
    ... ..
```

Running this function will result in an infinite loop, and it never gets to print "Important message!")

3

Why do we need recursion

- Many computational problems have well-understood recursive solutions
- Recursive solutions appear to be easy to understand and code
- These recursive solutions can often be converted to iterative solutions (using loops)
- But the iterative solutions are harder to understand and code
- Recursion can be a powerful tool to solve some problems

4

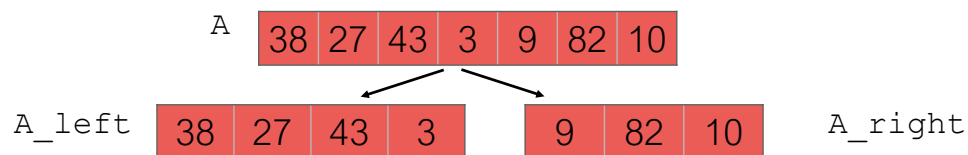
When to stop a recursion

- **Base case(s)**, usually given in conditional statements, do not lead to recursive calls
- Once a recursion reaches the base case(s), it stops

```
def my_function(n):  
    if n<=0:  
        print(n)  
    else:  
        print(n)  
        my_function(n-1)
```

5

Merge Sort

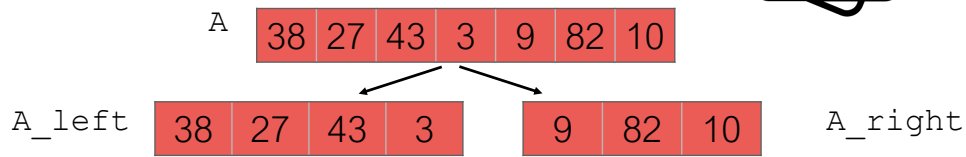


In order to sort a

- sort **A_left**
- sort **A_right**
- merge sorted **A_left** and **A_right**

6

The merge



sorted A_left [3, 27, 38, 43]

sorted A_right [9, 10, 82]

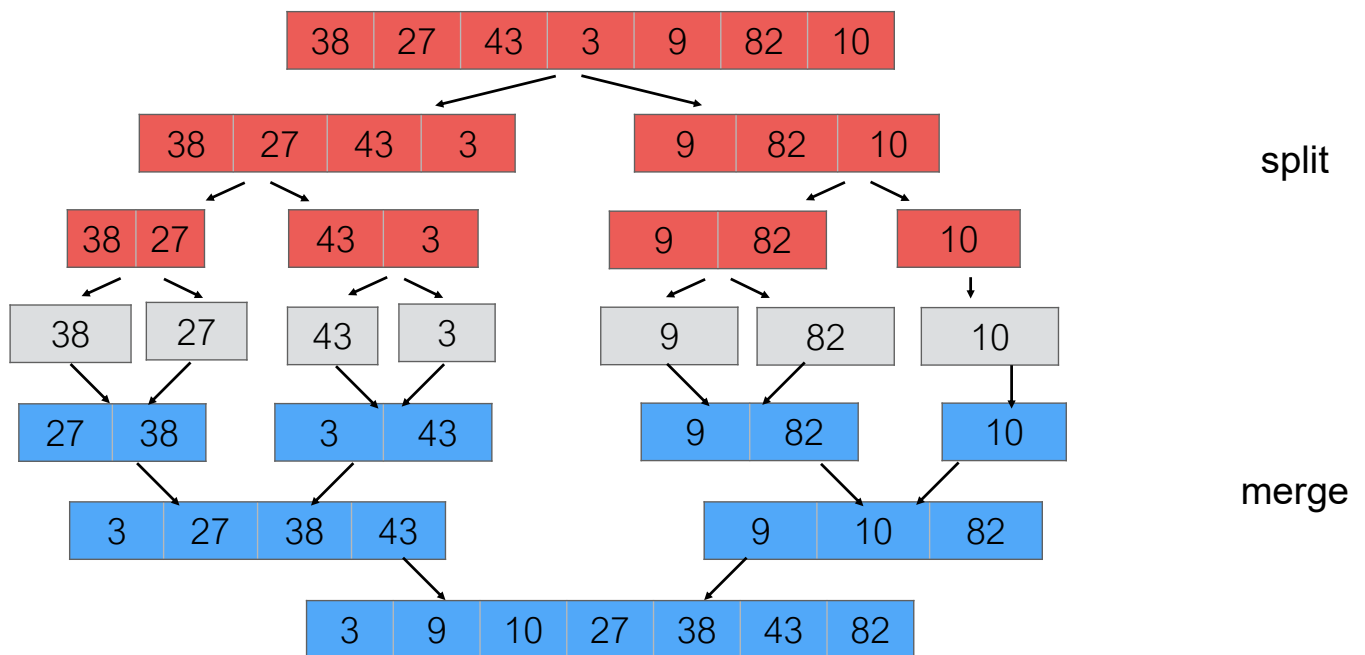
[3, 9, 10, 27, 38, 43, 82]

7

Merge Sort

1. Divide the n -element sequence to be sorted into two subsequences of $n/2$ elements each
2. Sort the two subsequences recursively using Merge-Sort
3. Merge the two sorted subsequences to produce the sorted answer

Merge Sort



9

Merge sort

Merge two sorted subsequences, $A[p \dots q]$ and $A[q + 1 \dots r]$

MERGE-SORT(A, p, r)

```

1  if  $p < r$ 
2       $q = \lfloor (p + r) / 2 \rfloor$ 
3      MERGE-SORT( $A, p, q$ )
4      MERGE-SORT( $A, q + 1, r$ )
5      MERGE( $A, p, q, r$ )

```

MERGE(A, p, q, r)

```

1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1 \dots n_1 + 1]$  and  $R[1 \dots n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 

```

Complexity of recursive algorithms

```
A(n) :  
    if n >= 1:  
        print n  
        A(n-1)
```

- Will $A(7)$ run longer than $A(5)$?
- Can the running time be determined based on n ?
- What is the relation between the size of the input to the function and the number of steps executed?

11

Recurrence relation

```
A(n) :  
    if n >= 1:  
        print n  
        A(n-1)
```

1. if $n = 0$

2. if $n \geq 1$

$$T(n) = c_1 \text{ for } n = 0$$

$$T(n) = c_2 + T(n - 1) \text{ for } n \geq 1$$

12

Complexity of recurrence relation

$$T(n) = c_1 \text{ for } n = 0$$

$$T(n) = c_2 + T(n - 1) \text{ for } n \geq 1$$

- How to transform the recurrence relation into a closed-form formula?
- How to get rid of the self-reference?
- Method - **expansion** or **substitution**

13

Substitution

$$T(n) = c_1 \text{ for } n = 0$$

$$T(n) = c_2 + T(n - 1) \text{ for } n \geq 1$$

$$T(x) = c_2 + T(x - 1)$$

$$T(n - 1) = c_2 + T(n - 2)$$

$$T(n) = c_2 + c_2 + T(n - 2)$$

$$= c_2 + c_2 + c_2 + T(n - 3)$$

$$= c_2 + c_2 + c_2 + c_2 + T(n - 4)$$

$$= c_2 + \dots + c_2 + T(0)$$

$$= c_2 + \dots + c_2 + c_1$$

$$= c_2 \times n + c_1$$

$$T(n) = \Theta(n)$$

14

Complexity of recurrence relation

Recurrence Relation	Complexity
$T(0) = c_1$ $T(n) = c_2 + T(n - 1)$	$\Theta(n)$

15

A second example

```
B(n) :  
    if n >= 1:  
        for i in range(n):  
            print n  
        B(n-1)
```

- What is the recurrence relation for the time function of $B(n)$?

16

```

B(n) :
    if n >= 1:
        for i in range(n):
            print n
        B(n-1)

```

$$T(n) = c_1 \quad \text{for } n = 0$$

$$T(n) = c_2 + c_3 \times n + T(n - 1) \quad \text{for } n \geq 1$$

17

A second example

```

B(n) :
    if n >= 1:
        for i in range(n):
            print n
        B(n-1)

```

$$T(n) = c_1 \quad \text{for } n = 0$$

$$T(n) = \Theta(n^2)$$

$$T(n) = c_2 + c_3 \times n + T(n - 1) \quad \text{for } n \geq 1$$

$$T(n) = c_2 \times n + c_3 \times \frac{(n+1)n}{2} + c_1$$

18

Complexity of recurrence relation

Recurrence Relation	Complexity
$T(0) = c_1$ $T(n) = c_2 + T(n - 1)$	$\Theta(n)$
$T(0) = c_1$ $T(n) = c_2 + c_3 \times n + T(n - 1)$	$\Theta(n^2)$